

OBJECT ORIENTED PROGRAMMING (25MC409)

I M.C.A II Semester

2025-26 EVEN SEMESTER

**Name of the Faculty : Dr. K. Gayatri
Assistant Professor, Department of CA
VFSTR (Deemed to be University)**

Institute Vision & Mission

VISION:

To evolve in to a centre of excellence in Science & Technology through creative and innovative practices in teaching-learning, towards promoting academic achievement and research excellence to produce internationally accepted, competitive and world class professionals who are psychologically strong & emotionally balanced imbued with social consciousness & ethical values.

MISSION:

To Provide High Quality academic programmes, training activities, research facilities and opportunities supported by continuous industry - institute interaction aimed at promoting employability, entrepreneurship, leadership and research aptitude among students and contribute to the economic and technological development of the region, state and nation.

Department Vision & Mission

VISION:

To emerge as a renowned center in the field of computer applications for providing high quality education and research to produce globally competent professionals with ethical values.

MISSION:

- **M1:** To provide high quality education with cutting-edge curriculum and learner-centered approach.
- **M2:** To establish the culture of innovation and research through industry-academia linkages.
- **M3:** To cultivate ethical and socially-conscious professionals with leadership qualities, who understand the global implications of technology.

PO' s of the Department

- **PO1 (Foundation Knowledge):** Apply knowledge of mathematics, programming logic and coding fundamentals for solution architecture and problem solving.
- **PO2 (Problem Analysis):** Identify, review, formulate and analyze problems for primarily focusing on customer requirements using critical thinking frameworks.
- **PO3 (Development of Solutions):** Design, develop and investigate problems with as an innovative approach for solutions incorporating ESG/SDG goals.
- **PO4 (Modern Tool Usage):** Select, adapt and apply modern computational tools such as development of algorithms with an understanding of the limitations including human biases.

PO' s of the Department

- **PO5 (Individual and Teamwork):** Function and communicate effectively as an individual or a team leader in diverse and multidisciplinary groups. Use methodologies such as agile.
- **PO6 (Project Management and Finance):** Use the principles of project management such as scheduling, work breakdown structure and be conversant with the principles of Finance for profitable project management.
- **PO7 (Ethics):** Commit to professional ethics in managing software projects with financial | aspects. Learn to use new technologies for cyber security and insulate customers from malware
- **PO8 (Life-long learning):** Change management skills and the ability to learn, keep up with contemporary technologies and ways of working.

PEO' s of the Department

- **PEO1:** Develop cutting-edge technical skills to succeed in career and as an entrepreneur.
- **PEO2 :** Pursue higher education and research with critical thinking and problem solving capabilities
- **PEO3:** Demonstrate ethical, collaborative, and leadership abilities in multi-disciplinary domains.

PSO's of the Department

- PSO1: To provide solutions for complex computational problems with requisite mathematical, programming languages and tools.
- PSO2: To design, develop and analyse software and related services using algorithms, web design, and Big Data Analytics for real time applications.

Module-2

Unit-1

PACKAGES: Creating, defining and accessing packages, importing packages.

EXCEPTION HANDLING: Concepts of exception handling, Types of exceptions, Built- in exceptions, Usage of try, catch, throw, throws and finally keywords, Custom Exception.

PACKAGES



- A package as the name suggests is a pack (group) of classes, interfaces and other packages.
- A java package is a group of similar types of classes, interfaces and sub-packages.

Types of packages in Java

- 1) **User defined package**: The package we create is called user-defined package.
- 2) **Built-in package**: The already defined packages in Java are known as built-in packages.

Advantage of Java Package

- 1) Java package is used to categorize the classes and interfaces so that they can be easily maintained.
- 2) Java package provides access protection.
- 3) Java package removes naming collision.



Java API (Application Programming Interface) packages (or) Predefined packages (or) Built-in packages:

Built-in packages:

1. java.lang - This package consists of classes that help in performing input out operations in the program. It also contains language support for data types and math operations. Whenever you create a class, this package is automatically imported.

2. java.io - This package contains classes that specialize in input and output operations only. One popular thing is the InputStreamReader class and the BufferedReader class.



3. **java.util** - This class contains basic utilities that can be useful while implementing LinkedLists, Trees, HashMaps, and so on. It also has a Scanner class for input-output operations to the program.
4. **java.applet** - This package contains necessary classes for creating applets.
5. **java.awt** - This package specializes in providing support for designing GUI elements in Java.
6. **java.net** - This package contains classes which help in performing network operations.



Creating a Package

- The “package” keyword is used to create a package in java.
- While creating a package, we should choose a name for the package and include a package statement along with that name at the top of every source file that contains the classes, interfaces and so on that we want to include in the package
- The package statement should be the first line in the source file.
- There can be only one package statement in each source file, and it applies to all types in the file



Importing and Accessing Package

The “import” keyword is used to make the classes and interface of another package accessible to the current package.

Two important ways to access the package from outside the package.

1. `import package.*;`

If you use `package.*` then all the classes and interfaces of this package will be accessible

2. `import package.classname;`

If you import `package.classname` then only declared class of this package will be accessible.

Example 1:

```
import mypack.number;  
class pack  
{  
    public static void main(String[] args)  
    {  
        number obj=new number();  
        obj.add(3,4);  
    }  
}
```

Compilation : javac -d . Filename.java

Running: java packagename.filename

```
package mypack;
    public class number
    {
        public void add (int a,int b)
        {
            System.out.println("Sum="+ (a+b));
        }
    }

package mypack;
public class number1
{
    public void sub(int a,int b)
    {
        System.out.println("sub="+ (a-b));
    }
}
```

Defining a Package:

To define a package, place “package” command as the first statement in the Java source file. So, that any class declared within that file will belong to the specified package. The syntax of package creation is as follows.

Syntax: `package pack-name;`

where pack-name is the name of the package.

Example:

```
package mypack;  
public class number  
{  
    public void add (int a,int b)  
    {  
        System.out.println("Sum="+ (a+b));  
    }  
}
```

The class that is defined in the package must be start with the public access modifier. So, that it can be accessible by any another of them. If it is not public, it is possible to access only in that package.

```
import mypack.number;  
import mypack.number1;  
    class pack  
    {  
    public static void main(String[] args)  
    {  
    number obj=new number();  
    obj.add(3,4);  
    number1 obj1=new number1();  
    obj1.sub(6,2);  
    }  
    }
```

Sub Packages:

It is also possible to create sub packages for the main package.

Syntax: `package pack1.pack2;`

where pack1 is the main package and pack2 is the sub package.

Example:

```
package pack1;  
public class x  
{  
    public void show()  
    {  
    }
```

```
        System.out.println("Super");
    }
}
package pack1.pack2;
public class y
{
    public void display()
    {
        System.out.println("Sub");
    }
}
```

```
import pack1.x;  
import pack1.pack2.y;  
class che  
{  
    public static void main(String[] args)  
    {  
        x obj=new x();  
        obj.show();  
        y obj1=new y();  
        obj1.display();  
    }  
}
```

EXCEPTION HANDLING

- An exception is an abnormal event that rises during the execution of a program and disturbs the normal flow of instructions i.e., exception is a run-time error.

Consider the following example:

```
class hello
{
    public static void main(String[] args)
    {
        int a=10,b;
        b=a/0;
        System.out.println("value="+b);
    }
}
```

- When we try to compile this program, it does not produce any errors. But when it runs the program, it produces a message as "java.lang.ArithmeticException: / by zero". This exception error message is displayed and the program terminates without executing the remaining statements.
- The core advantage of exception handling is to maintain the normal flow of the application. An exception normally disrupts the normal flow of the application that is why we use exception handling.

Let's take a scenario:

statement 1;

statement 2;

statement 3;

statement 4;

statement 5;//exception occurs

statement 6;

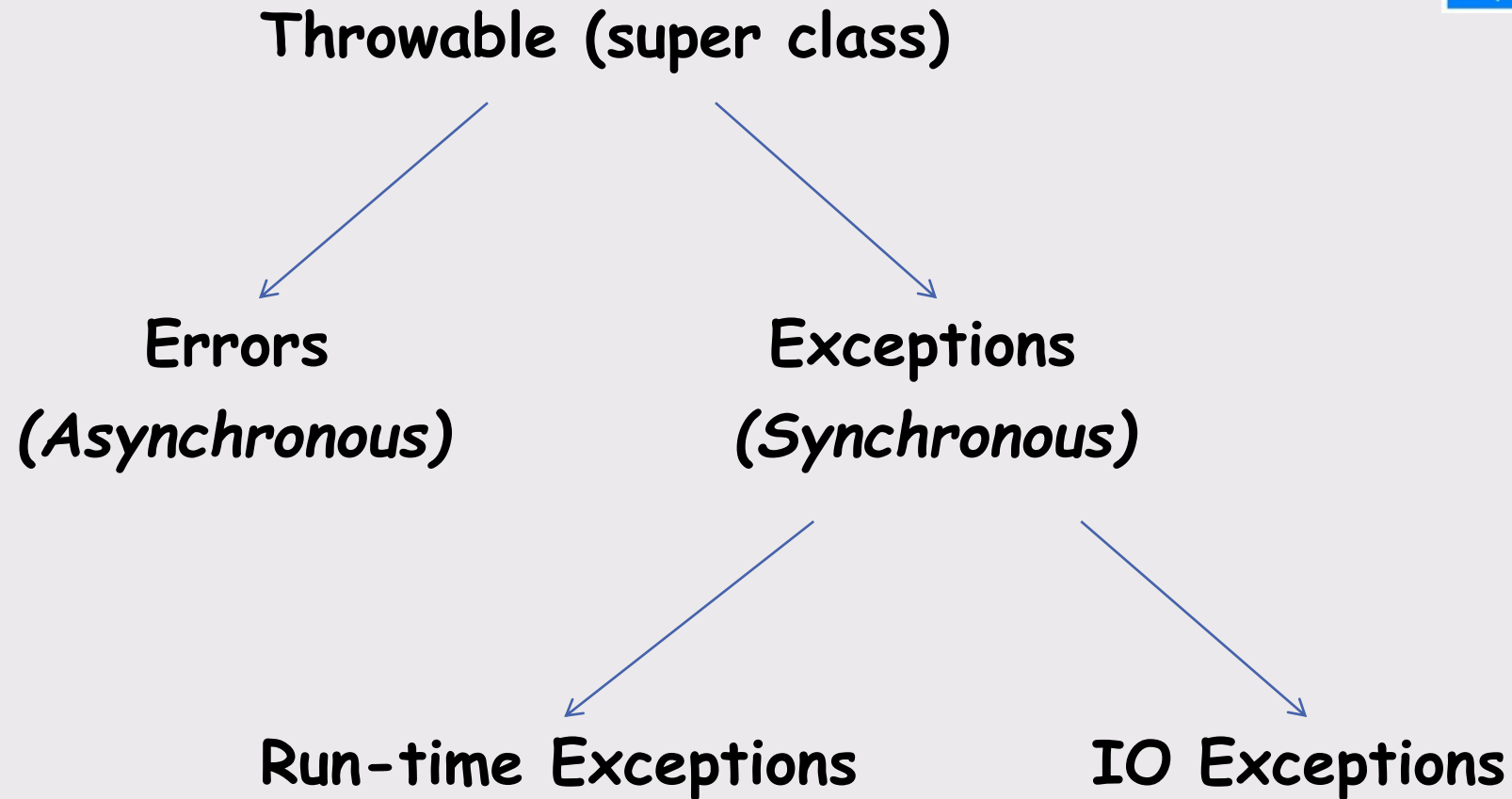
statement 7;

statement 8;

statement 9;

statement 10;

- Suppose there are 10 statements in your program and there occurs an exception at statement 5, the rest of the code will not be executed i.e. statement 6 to 10 will not be executed. If we perform exception handling, the rest of the statement will be executed. That is why we use exception handling in Java.
- All exception types are subclasses of built-in class Throwable. It is a super class. The class is available in java.lang package. It contains errors and exceptions.



- There are mainly two types of exceptions:
 - Checked
 - Unchecked.
- Here, an error is considered as the unchecked exception. According to Oracle, there are three types of exceptions:
- **Checked Exception**
 - ❑ The classes which directly inherit Throwable class except RuntimeException and Error are known as checked exceptions e.g. IOException, SQLException etc.
 - ❑ Checked exceptions are checked at compile-time.

- **Unchecked Exception**

- ❑ The classes which inherit Runtime Exception are known as unchecked exceptions.
- ❑ e.g. `ArithmeticException`, `NullPointerException`, `ArrayIndexOutOfBoundsException` etc.
- ❑ Unchecked exceptions are not checked at compile-time, but they are checked at runtime.

Error

Error is irrecoverable e.g. `OutOfMemoryError`, `VirtualMachineError`, `AssertionError` etc.

Java defines several exception classes inside the java.lang package. These are all subclass of Run-time Exceptions. The following are the some of the examples of exceptions and these are unchecked exceptions because the compiler does not check to see if a method handles or throws these exceptions.

Exception	Meaning
ArithmeticException	Arithmetic error, such as divide-by-zero
ArrayIndexOutOfBoundsException	Array index is out-of-bounds
IllegalArgumentException	Illegal argument used to invoke a method

IllegalThreadStateException	Requested operation not compatible with current thread state
IndexOutOfBoundsException	Some type of index is out-of-bounds
NegativeArraySizeException	Array created with a negative size
NullPointerException	Invalid use of a null reference
NumberFormatException	Invalid conversion of a string to a numeric format
StringIndexOutOfBoundsException	Attempt to index outside the bounds of a string

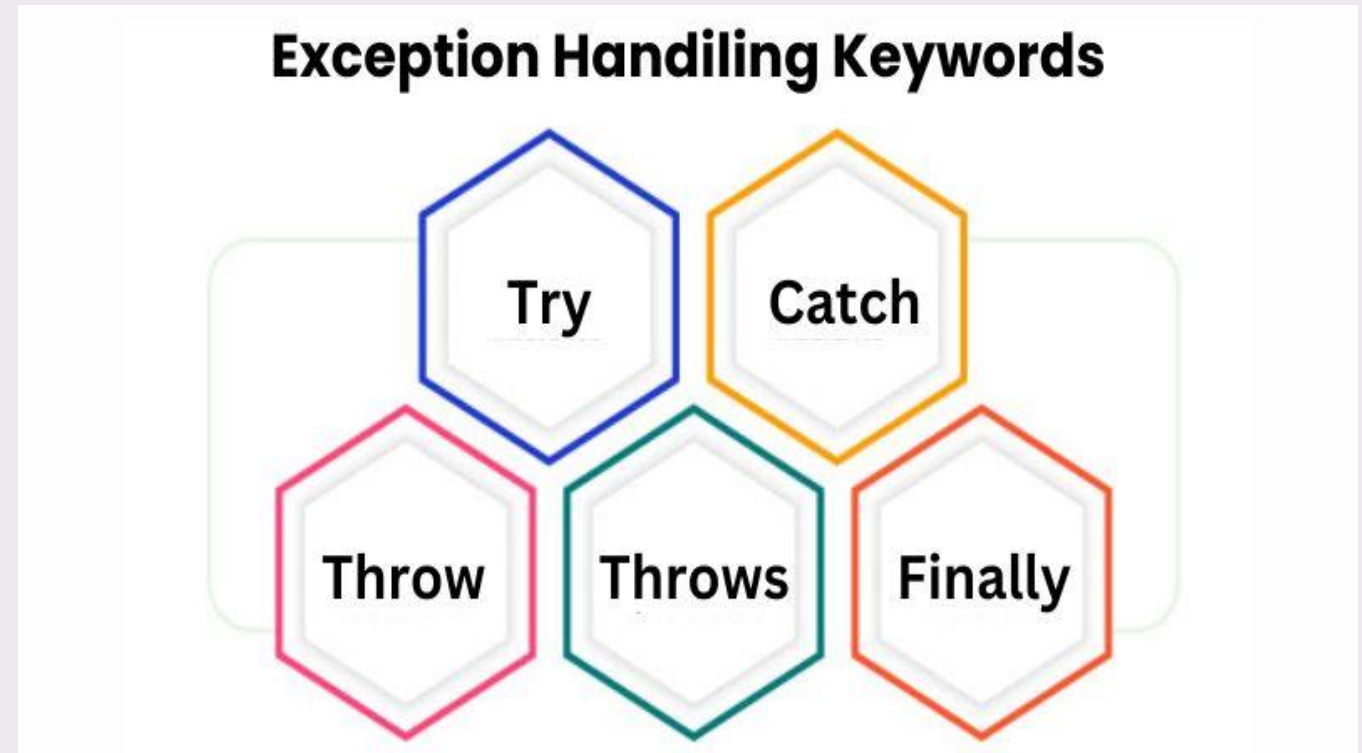
- `java.lang` package contains some another type of exceptions that does not handle itself. Here, the compiler checks that what is to be done when this arise, because of this checking that they are called checked exceptions. Examples for checked exceptions are

Exception	Meaning
<code>ClassNotFoundException</code>	Class not found
<code>IllegalAccessException</code>	Access to a class is denied
<code>InterruptedException</code>	One thread has been interrupted by another thread
<code>NoSuchMethodException</code>	A requested method does not exist

Java exception handling is managed through five keywords.

They are

- 1) try
- 2) catch
- 3) throw
- 4) throws
- 5) finally



Keyword	Description
try	The "try" keyword is used to specify a block where we should place exception code. The try block must be followed by either catch or finally. It means, we can't use try block alone.
catch	The "catch" block is used to handle the exception. It must be preceded by try block which means we can't use catch block alone. It can be followed by finally block later.
finally	The "finally" block is used to execute the important code of the program. It is executed whether an exception is handled or not.

Throw	The "throw" keyword is used to throw an exception.
throws	The "throws" keyword is used to declare exceptions. It doesn't throw an exception. It specifies that there may occur an exception in the method. It is always used with method signature.

try block:

The statements that are produces exception are identified in the program and the statements are placed with in a try block

Syntax:

```
try
{
    // block of code to monitor for errors
}
```

If an exception occurs within the try block, the appropriate exception handler (catch block) associated with the try block handles the exception immediately.

catch block:

The catch block is used to process the exception raised. The catch block is placed immediately after the try block.

Syntax:

```
catch (ExceptionType exob)
{
    // exception handler for exception type
}
```

finally block:

This block is placed after the last catch block. This block is executed regardless of whether or not an exception is raised.

Syntax:

```
finally
{
    // block of code to be executed before of try block ends
}
```

1. The control enters into the try block and executes the statements. If an exception is raised, the exception throws and it caught by exception handler catch block. And the particular catch block statements are executed. The control never backs again to the try block. If no exceptions raised, catch blocks are not executed the control directly passed to the finally block.
2. No statements are placed between try block and catch block.

```
class ex11 {  
    public static void main(String[] args) {  
        try  
        {  
            int a=10,b;  
            b=a/0;  
            System.out.println("b="+b);  
        }  
        catch(ArithmeticException e)  
        {  
            System.out.println("Exception raised");  
        }  
        System.out.println("Quit");  
    }  
}
```

**O/P: Exception raised
Quit**

```
class ex12 {  
    public static void main(String[] args) {  
        int x[]=new int[10];  
        try  
        {  
            x[20]=100;  
        }  
        catch(ArrayIndexOutOfBoundsException e)  
        {  
            System.out.println("Exception caught");  
        }  
        finally  
        {  
            System.out.println("Not possible to print");  
            System.out.println("Quit");  
        }  
    }  
}
```

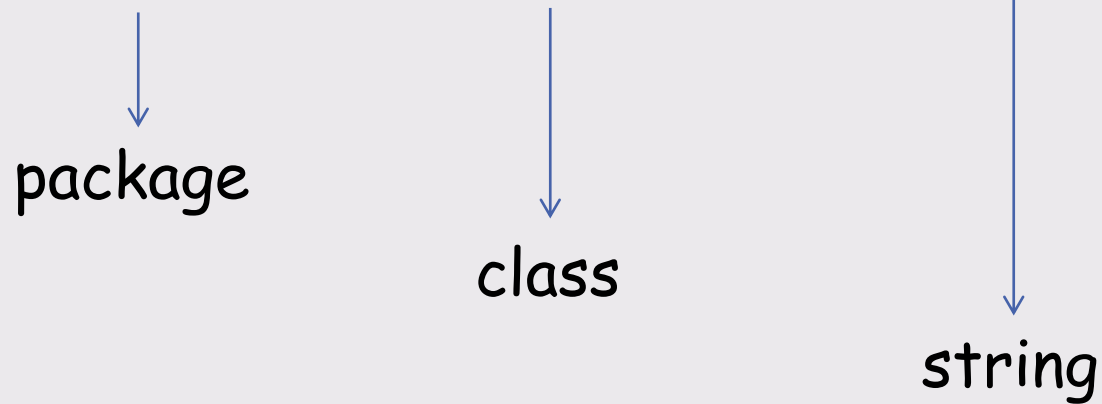
Methods of Exception Object

a) **String getMessage()**: It returns the descriptive string stored in an exception.

Example: / by zero

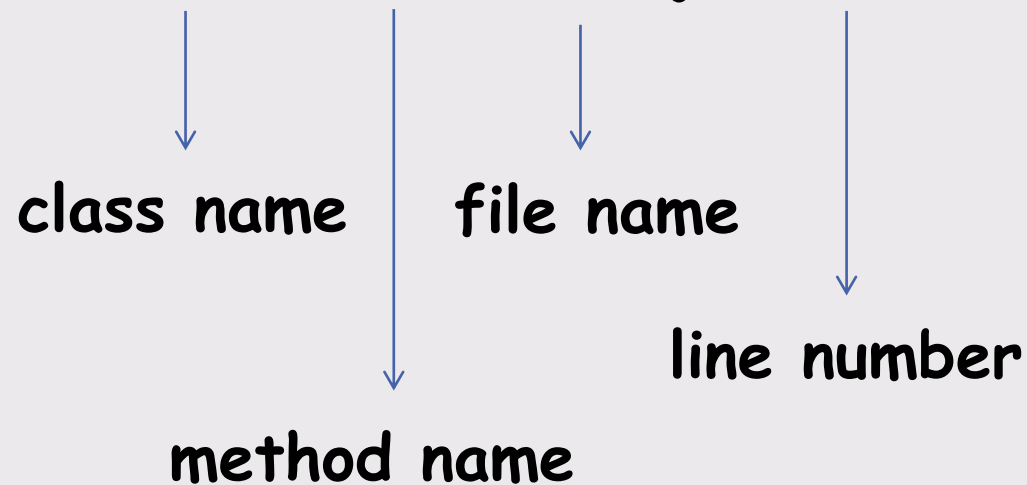
b) **String toString()**: It returns an error message with the class name of the exception the descriptive string to be stored in the exception.

Example: java.lang.ArithmeticException: / by zero



c) **void printStackTrace()**: This is the standard output error stream. It displays the error message with the class name of the exception the descriptive string to be stored in the exception along with the line number and name of the program.

Example: `java.lang.ArrayIndexOutOfBoundsException`
`at ex12.main (ex12.java:8)`



```
import java.text.DecimalFormat;
class n
{
    public static void main(String[] args)
    {
        try
        {
            double x=25.6776;
            DecimalFormat d=null;
            System.out.println(d.format(x));
        }
        catch(NullPointerException e)
        {
            System.out.println(e.getMessage());
            System.out.println(e.toString());
            e.printStackTrace();
        }
    }
}
```

o/p:
null

Java.lang. NullPointerException
Java.lang. NullPointerException
at n.main<n.java:10>

Multiple Catch Statements

- Multiple catch clauses handle the situation where more than one exception could be raised by a single piece of code. In such situations specify two or more catch blocks, each specify different type of exception.

Rule: At a time only one Exception is occurred and at a time only one catch block is executed.

Rule: All catch blocks must be ordered from most specific to most general i.e. catch for `ArithmeticException` must come before catch for `Exception`

```
class cat
{
    public static void main(String[] args)

    {
        try
        {
            int a=args.length;
            System.out.println("a="+a);
            int b=42/a;
            int c[]={1};
            c[42]=99;
        }
    }
}
```

```
catch(ArithmeticException e)
{
    System.out.println(e.getMessage());
}
catch(ArrayIndexOutOfBoundsException e)
{
    System.out.println(e.getMessage());
}
}
```

Java Nested try block

- The try block within a try block is known as nested try block in java.
- Sometimes a situation may arise where a part of a block may cause one error and the entire block itself may cause another error. In such cases, exception handlers have to be nested.

Syntax:

```
try
{
    statement 1;
    statement 2;
    try
    {
        statement 1;
        statement 2;
    }
    catch(Exception e)
    {
    }
}
catch(Exception e)
{
}
```

```
class ntry
{
    public static void main(String[] args)
    {
        try
        {
            int a=args.length;
            //int b=42/a;
            int d[]={1};
            d[50]=88;
            System.out.println("a="+a);
            try
            {
                if(a==1)
                a=a/(a-a);
                if(a==2)
                {
```

```
int c[]={1};
    c[42]=99;
}
}
catch(ArrayIndexOutOfBoundsException e)
{
    System.out.println("Array Exception:"+e);
}
}
catch(ArithmeticException e)
{
    System.out.println("Arithmetic exception:"+e);
}
}
}
```

Java finally block

- Java finally block is a block that is used to *execute important code* such as closing connection, stream etc.
- Java finally block is always executed whether exception is handled or not.
- Java finally block follows try or catch block.
- *Note: If you don't handle exception, before terminating the program, JVM executes finally block(if any).*

Let's see the java finally example where exception doesn't occur.

```
class TestFinallyBlock
{
    public static void main(String args[])
    {
        try{
            int data=25/5;
            System.out.println(data);
        }
        catch(NullPointerException e){System.out.println(e);}
        finally{
            System.out.println("finally block is always executed");
        }
        System.out.println("rest of the code...");
    }
}
```

o/p:

5

finally block is always executed
rest of the code....

```
class TestFinallyBlock1{  
    public static void main(String args[]){  
        try{  
            int data=25/0;  
            System.out.println(data);  
        }  
        catch(NullPointerException e) { System.out.println(e); }  
        finally{  
            System.out.println("finally block is always executed");  
        }  
        System.out.println("rest of the code...");  
    }  
}
```

o/p:

finally block is always executed

Exception in thread main

java.lang.ArithmeticException:/ by zero

Let's see the java finally example where exception occurs and handled.

```
public class TestFinallyBlock2{  
    public static void main(String args[]){  
        try{  
            int data=25/0;  
            System.out.println(data);  
        }  
        catch(ArithmeticException e) {System.out.println(e);}  
        finally{  
            System.out.println("finally block is always executed");  
        }  
        System.out.println("rest of the code...");  
    }  
}
```

o/p:

Exception in thread main

java.lana.ArithmeticException:/ by zero

finally block is always executed

Rest of the code.....

Rule: For each try block there can be zero or more catch blocks, but only one finally block.

- Note: The finally block will not be executed if program exits(either by calling `System.exit()` or by causing a fatal error that causes the process to abort).

throw statement

- All previous exceptions are catching that are thrown by the java run time system.
- It is also possible to create for a program that throws an exception explicitly, using the "throw" statement.

Syntax: throw throwable-instance;

- Here, throwable-instance must be an object type of **Throwable** class or subclass of **Throwable**.
- There are two ways to obtain a **Throwable** object.
 - ☐ Using a parameter into a catch clause
 - ☐ Creating one with the new operator

```
class ntry2
{
    public static void main(String[] args)
    {
        try
        {
            throw new ArithmeticException("hello");
        }
        catch(ArithmeticException e)
        {
            System.out.println(e.getMessage());
        }
    }
}
```

O/P:
hello

```
public class TestThrow1
{
    static void validate(int age)
    {
        if(age<18)
            throw new ArithmeticException("not valid");
        else
            System.out.println("welcome to vote");
    }
    public static void main(String args[])
    {
        validate(13);
        System.out.println("rest of the code...");
    }
}
```

Output:

```
Exception in thread main
java.lang.ArithmeticException:not valid
```

```
class t
{
    public t()
    {
        try
        {
            throw new NullPointerException("demo");
        }
        catch(NullPointerException e)
        {
            System.out.println("caught");
            throw e; //rethrow the exception
        }
    }
}
```

```
class ntry2
{
    public static void main(String[] args)
    {
        try
        {
            t obj=new t();
        }
        catch(NullPointerException e)
        {
            System.out.println("Recaught");
        }
    }
}
```

O/P:
cauht
Recaught

Java throws keyword

- The **Java throws keyword** is used to declare an exception.
- It gives an information to the programmer that there may occur an exception so it is better for the programmer to provide the exception handling code so that normal flow can be maintained.
- Exception Handling is mainly used to handle the checked exceptions.
- If there occurs any unchecked exception such as `NullPointerException`, it is programmers fault that he is not performing check up before the code being used.

```
type method-name(parameter-list) throws exception-list
{
    // method body
}
```

Let's see the example of java throws clause which describes that checked exceptions can be propagated by throws keyword.

```
import java.io.IOException;
class Testthrows1
{
    void m()throws IOException
    {
        throw new IOException("device error");//checked exception
    }
    void n()throws IOException
    {
        m();
    }
}
```

```
void p()  
{  
    try  
    {  
        n();  
    }  
  
    catch(Exception e)  
    {  
        System.out.println("exception handled");  
    }  
  
    public static void main(String args[])  
    {  
        Testthrows1 obj=new Testthrows1();  
        obj.p();  
        System.out.println("normal flow...");  
    }  
}
```

Output:
exception handled
normal flow...

Rule: If you are calling a method that declares an exception, you must either catch or declare the exception.

There are two cases:

- ❖ **Case1:** You caught the exception i.e. handle the exception using try/catch.
- ❖ **Case2:** You declare the exception i.e. specifying throws with the method.

Case1: You handle the exception

In case you handle the exception, the code will be executed fine whether exception occurs during the program or not.

```
import java.io.*;
class M
{
    void method()throws IOException
    {
        throw new IOException("device error");
    }
}
```

```
public class Testthrows2{  
    public static void main(String args[])  
    {  
        try  
        {  
            M m=new M();  
            m.method();  
        }  
        catch(Exception e)  
        {  
            System.out.println("exception handled");  
            System.out.println("normal flow...");  
        }  
    }  
}
```

Output:
exception handled
normal flow...

Case2: You declare the exception

- A. In case you declare the exception, if exception does not occur, the code will be executed fine.
- B. In case you declare the exception if exception occurs, an exception will be thrown at runtime because throws does not handle the exception.

A. Program if exception does not occur

```
import java.io.*;
class M
{
    void method()throws IOException{
        System.out.println("device operation performed");
    }
}
```

```
}  
}  
class Testthrows  
{  
    public static void main(String args[])throws IOException{ //declare exception  
        M m=new M();  
        m.method();  
  
        System.out.println("normal flow...");  
    }  
}
```

Output:
device operation performed
normal flow...

B. Program if exception occurs

```
import java.io.*;
class M
{
    void method()throws IOException
    {
        throw new IOException("device error");
    }
}
class Testthrows
{
    public static void main(String args[])throws IOException    //declare exception
    {
        M m=new M();
        m.method();

        System.out.println("normal flow...");
    }
}
```

Output:
Runtime Exception

Difference between throw and throws in Java

There are many differences between throw and throws keywords.

A list of differences between throw and throws are given below:

No.	throw	throws
1)	Java throw keyword is used to explicitly throw an exception.	Java throws keyword is used to declare an exception.
2)	Checked exception cannot be propagated using throw only.	Checked exception can be propagated with throws.
3)	Throw is followed by an instance.	Throws is followed by class.
4)	Throw is used within the method.	Throws is used with the method signature.
5)	You cannot throw multiple exceptions.	You can declare multiple exceptions e.g. public void method()throws IOException,SQLException.

Difference between final, finally and finalize

There are many differences between final, finally and finalize.

A list of differences between final, finally and finalize are given below:

No.	final	finally	finalize
1)	Final is used to apply restrictions on class, method and variable. Final class can't be inherited, final method can't be overridden and final variable value can't be changed.	Finally is used to place important code, it will be executed whether exception is handled or not.	Finalize is used to perform clean up processing just before object is garbage collected.
2)	Final is a keyword.	Finally is a block.	Finalize is a method.